

Resampling Methods

Resampling methods involve repeatedly drawing sample from a data set and refitting a model of interest on each sample in order to obtain additional information about the fitted model. For example, in order to estimate the variability of a linear regression fit, we can repeatedly draw different samples from the training data, fit a linear regression to each new sample, and then examine the extent to which the resulting fits differ. Such an approach may allow us to obtain information than would not be available from fitting the model only once using the original sample. Resampling approaches can be computationally expensive, because they involve fitting the same statistical method multiple times using different subsets of the data. However, due to advances in computing power, the computational requirements of resampling methods generally are not prohibitive.

The Bootstrap

The bootstrap is a method of numerical resampling used to estimate properties of an unknown distribution (most often the confidence interval of a parameter).

Suppose that we wish to find the confidence interval (CI) for a parameter θ for some distribution $f(x|\theta)$. We take a sample from the distribution and use some estimator with the data.

For example, the estimator for the population mean is

$$\hat{\mu} = \frac{\sum_{i=1}^n x_i}{n}$$

If the data comes from a normal distribution, then the confidence interval can be estimated as

$$CI = \bar{X} \pm t_{\frac{\alpha}{2}} \frac{s}{\sqrt{n}}$$

Similarly, the estimator for a binomial proportion p is

$$\hat{p} = \frac{\text{number of successes}}{n}$$

If the data is binomially distributed and n is large, then we can estimate the confidence interval as

$$CI = \hat{p} \pm t_{\frac{\alpha}{2}} \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}}$$

Calculating these confidence intervals requires that we know the distribution of the data or at least are willing to assume it. If we don't know the distribution for the data and are unwilling to assume one, then we can't estimate confidence intervals using simple formulas. Sometimes even when we do know the distribution of the data there still isn't a simple formula for calculating the confidence interval.

Repeated Sampling

However, suppose that we can do the following

1. Repeat the experiment a large number c times, each time collecting new data.
2. For each data set, calculate a parameter estimate $\hat{\theta}$ and store it.
3. Then a 95% CI for the parameter would be

$$Z_1 < \hat{\theta} < Z_2$$

where Z_1 =point such that portion 0.025 of the $\hat{\theta}$ values fall below

Z_2 =point such that portion 0.025 of the $\hat{\theta}$ values fall above.

As $c \Rightarrow \infty$, the CI estimated this way converges to the true CI.

Problem: We of course in most situations can't really repeat the experiment a large number of times.

Solution: Bootstrap resampling.

Bootstrap Procedure

Suppose that we have an iid sample $x_1, x_2, \dots, x_n$ from $f(x|\theta)$ and we want to estimate θ .

We take c samples **with replacement** from $x_1, x_2, \dots, x_n$. Thus, some x_i values will occur multiple times and some won't occur at all.

We can use this collection of values from $x_1, x_2, \dots, x_n$ to approximate a collection of samples from $f(x|\theta)$.

intuition: $x_1, x_2, \dots, x_n$ come from $f(x|\theta)$. For a sufficiently large n , the cdf $F(x|\theta)$ is approximated by

$$F(X|\theta) = P(x \leq X) \approx \frac{n_x}{n}$$

where n_x is the number of x_i that are less than X .

Thus, the $x_1, x_2, \dots, x_n$ approximate $f(x|\theta)$ and sampling from $x_1, x_2, \dots, x_n$ approximates sampling from $f(x|\theta)$.

A great deal of theoretical and simulation work have shown that this procedure works well for sufficiently large n and c and then the *bootstrap CI* $\approx$ *true CI*.

Example #1- CI for the median of a gamma distribution

The data set below consists of 10 values from a gamma distribution. Use it to find a confidence interval for the median of the gamma.

3.2 3.6 3.8 4.1 3.6 2.9 2.1 3.3 2.7 3.9

There is not any easy formula for this CI and thus finding it directly would be difficult. However, we can do it easily using the bootstrap. The procedure is

```
loop over bootstrap replicates 1 to R
{
take a sample of size 10 from the data set with replacement
calculate the median of the bootstrap sample and store the value
}
```

The CI is estimated as the middle 95% of the bootstrap values

Here is the code:

```
d1=c(3.2, 3.6, 3.8, 4.1, 3.6, 2.9, 2.1, 3.3, 2.7, 3.9) #data set
numsamp=10000; #number of bootstrap samples
resamp=numeric(numsamp) #makes numeric vector of length numsamp

#the sample(d1,replace=T) command takes a sample with replacement from the values in d1
for(i in 1:numsamp) resamp[i]=median(sample(d1,replace=T))
sd(resamp) #standard error of median
```

```
## [1] 0.2435547
```

```
quantile(resamp,prob=c(0.025,0.975)) #CI for median
```

```
## 2.5% 97.5%
```

```
## 2.9 3.8
```

Deriving an estimator for the median of a gamma would be difficult. It is trivial to do with the bootstrap. Of course, we haven't explored how well the bootstrap CIs work. You will explore this in the next assignment.

Let's look at another example:

Example #2: Compare the bootstrap and theoretical CI's for the mean of normally distributed data

Next, we will calculate CIs for the mean of a normal distribution. Since we know how to calculate this CI directly, we can use this to compare the bootstrap CI to the theoretical CI.

Here is code to do the bootstrap sampling:

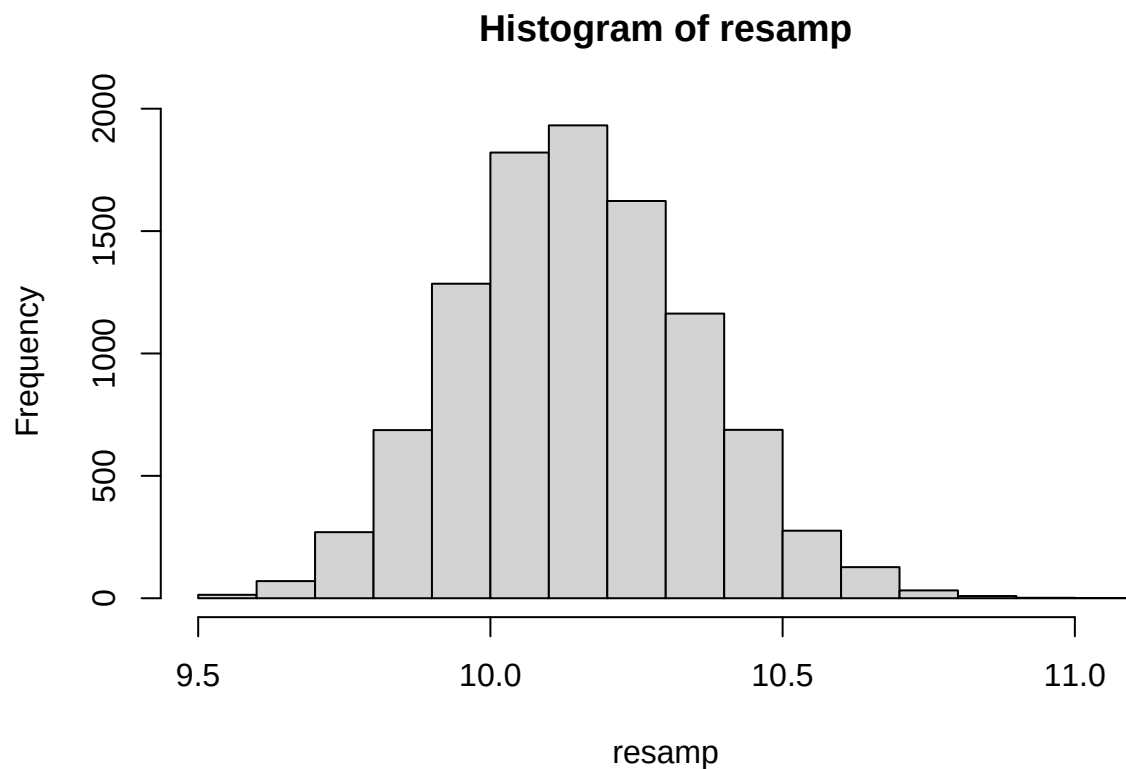
```
library(MASS)

set.seed(56) #sets seed for random number generator (if you set the same seed, you will
             #get the same random numbers)
d2<-c(rnorm(10,mean=10)) #sample data

numsamp=10000;#number of bootstrap replicates
resamp=numeric(numsamp) #makes numeric vector of length numsamp

for(i in 1:numsamp) resamp[i]=mean(sample(d2,replace=T)) #here is the resampling step

hist(resamp) #display results
```



The bias is zero:

```
mean(resamp-mean(d2)) #bias is zero
```

```
## [1] 0.0009541513
```

That is, the mean of the bootstrap parameter estimates is the same as the sample mean (which we know is an unbiased estimator of the population mean)

Is

The bootstrap-estimated standard error is the sample standard deviation of the bootstrap estimates. Compare this to the usual standard error of the mean:

```
sd(resamp) #standard dev of mean. Compare to
```

```
## [1] 0.2019595
```

```
sd(d2)/sqrt(10) #usual standard error of the mean
```

```
## [1] 0.2143074
```

They are very similar.

Finally, calculate the bootstrap CI and compare to the standard CI:

```
quantile(resamp,prob=c(0.025,0.975)) #CI for mean
```

```
##      2.5%      97.5%
##  9.771277 10.562479
```

```
mean(d2)+qt(0.025,length(d2)-1)*(sd(d2)/sqrt(length(d2))) #theoretical values (very close, but
```

```
## [1] 9.665318
```

```
mean(d2)+qt(0.975,length(d2)-1)*(sd(d2)/sqrt(length(d2))) #bootstrap gives slightly bigger CI)
```

```
## [1] 10.63491
```

The bootstrap CI is slightly smaller.

While the mean has a known standard error, most other statistics don't. The bootstrap is especially useful when there is no known theoretical standard error. As an example, let's find the standard error and CI for median of the above data set:

Exercise #1:

Find bootstrap CI for the median of the data set d2.

Python Code

Here is Python code to find a confidence interval for the median of a sample from a normal distribution:

```
import pandas as pd
import numpy as np
import numpy.random as npr

np.random.seed(56)

n=10 #sample size
mu=10
R=1000 #number of bootstrap replicates

d1=npr.normal(loc=mu,size=n) #generate random data set

med_vec=np.empty(shape=R) #array to hold bootstrap estimates of median

#do bootstrap replicates. Loop over bootstrap samples.
for r in np.arange(R):
    med_vec[r]=np.median(np.random.choice(d1,size=n,replace=True))

#Find bootstrap CI
np.quantile(med_vec,np.array([0.025,0.975]))
```

```
Out[22]: array([ 9.34241288, 10.59365816])
```

Sample Size Requirements for the Bootstrap

You will explore the performance of the bootstrap as a function of sample size in the homework. Generally, the bootstrap is not highly accurate for smaller sample sizes. That is, bootstrap confidence intervals will not tend to attain their nominal coverage at low sample sizes. For example, a 95% confidence interval will actually contain the population mean less than 95% of the time.

Unfortunately, there is not any way to know exactly what the true coverage of a bootstrap confidence interval will be. It will depend on sample size, the true distribution of the data, and the particular data points in your sample. Don't expect high accuracy when sample sizes are low (e.g. <20). It is often recommended to not use the bootstrap for sample sizes less than 10.

boot function

R has a built-in bootstrap functions that will be more computationally efficient. It is slightly complicated to use until you understand how it works. You need to define a wrapper function of the form `func(d,i)`, where `d` is the data set and `i` is a vector that defines the bootstrap sample. The function should return your desired test statistic applied to the bootstrap sample defined by `d[i]`.

In the example below, we will return to the data set `d2` that we used in example 2 above. We define the function `fm` that is the wrapper function needed by `boot`. We then use this with the `boot` function.

```
library(boot)

set.seed(56) #sets seed for random number generator (if you set the same seed, you will get the same ra
d2<-c(rnorm(10,mean=10)) #sample data

##boot
##boot.ci

fm<-function(d,i) mean(d[i]) #the stat argument to boot needs a function
                        #that takes dataset d and calculates the test
                        #statisic for the bootstrap sample defined by vector i

bd2<-boot(d2,fm,10000)
bd2

##
## ORDINARY NONPARAMETRIC BOOTSTRAP
##
##
## Call:
## boot(data = d2, statistic = fm, R = 10000)
##
##
## Bootstrap Statistics :
##      original      bias    std. error
## t1* 10.15011 0.0009541513  0.2044853

head(bd2$t) # is the bootstrap values (head() shows the first few values)

##           [,1]
```

```
## [1,] 10.01659
## [2,] 10.40069
## [3,] 10.52221
## [4,] 10.43119
## [5,] 10.67373
## [6,] 10.20032
```

The `boot` function outputs the bootstrap values of the test statistics. The `boot.ci` function takes the output of `boot` and calculates the CI:

```
boot.ci(bd2) #boot.ci takes output from boot() as input. It gives confidence intervals
```

```
## Warning in boot.ci(bd2): bootstrap variances needed for studentized intervals
```

```
## BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
## Based on 10000 bootstrap replicates
##
## CALL :
## boot.ci(boot.out = bd2)
##
## Intervals :
## Level      Normal              Basic
## 95%   ( 9.75, 10.55 )   ( 9.74, 10.54 )
##
## Level      Percentile          BCa
## 95%   ( 9.76, 10.56 )   ( 9.79, 10.60 )
## Calculations and Intervals on Original Scale
```

```
quantile(bd2$t,prob=c(0.025,0.975)) #note that this is the same as the percentile CI given by boot.ci
```

```
##      2.5%      97.5%
## 9.764893 10.561077
```

This gives four different CIs:

1. The CI based on the normal approximation.
2. The “basic” bootstrap interval. This is based on the distribution of Z values for the test statistic in the bootstrap sample. This method is known to be somewhat erratic in practice.
3. The percentile CI. This is the method that we have been using. Note that the result is the same as the CI that we found in example 1. This is better than the basic bootstrap CI, but also is sometimes erratic (especially for small samples).
4. The “adjusted” bootstrap percentile method. This is an improved method that gives the best results (see e.g. the book by Efron and Tibshirani).

Exercise #2

Use the `boot` function to find a bootstrap CI for the median for dataset `d2`.

Parametric Bootstrap

A parametric bootstrap is used when we know (or are willing to assume) the distribution of the data. In this case, we generate the bootstrap sample directly from the distribution instead of from the data. For example, if we knew that the above data came from a normal distribution, the parametric bootstrap procedure would be

1. estimate parameters of the normal distribution from the data
2. generate bootstrap samples from the normal distribution with these parameters

The parametric bootstrap is useful when the data is coming from some complicated distribution that we are able to simulate, but don't have a direct way to conduct inference.

Example #1 Parametric bootstrap for the mean of a normal distribution

The code below shows the parametric bootstrap calculation for the mean of normally distributed data. In each bootstrap replicate the steps are

1. generate data set from a normal distribution with mean `md1` and standard deviation `sd1`.
2. Calculate the mean and store the value.

The CI is the middle 95% of the stored values.

```
set.seed(56) #sets seed for random number generator
d2<-rnorm(10,mean=10) #sample data

numsamp<-10000
md1<-mean(d2) #estimate mean and variance of distribution
sd1<-sd(d2)
bootsamp<-numeric(numsamp) #vector to hold bootstrap values

#In each replicate, 1) generate data set from a normal distribution with mean md1 and standard deviation sd1
#2) Calculate mean and store value.
for(j in 1:numsamp) bootsamp[j]<-mean(rnorm(10,mean=md1,sd=sd1))

mean(bootsamp)-md1
```

```
## [1] -0.001219065
```

```
sd(bootsamp)
```

```
## [1] 0.2130805
```

```
quantile(bootsamp,prob=c(0.025,0.975)) #CI for mean
```

```
##      2.5%      97.5%
##  9.739328 10.571424
```

This is very similar to what we found above.

Note that there is not a reason that we would use a bootstrap to calculate a CI for the normal mean since there is a formula for this. The above is simply a demonstration of the procedure for a parametric bootstrap.

Parametric bootstrap with `boot()` function

The parametric bootstrap can also be implemented with the `boot` command

For the parametric bootstrap it is necessary for the user to specify how the resampling is to be conducted. The best way of accomplishing this is to specify the function `ran.gen` which will return a simulated data set from the observed data set and a set of parameter estimates specified in `mle` (see `?boot`). From the help page:

This function is used only when `sim = "parametric"` when it describes how random values are to be generated. It should be a function of two arguments. The first argument should be the observed data and the second argument consists of any other information needed (e.g. parameter estimates). The second argument may be a list, allowing any number of items to be passed to `ran.gen`. The returned value should be a simulated data set of the same form as the observed data which will be passed to `statistic` to get a bootstrap replicate. It is important that the returned value be of the same shape and type as the original dataset. If `ran.gen` is not specified, the default is a function which returns the original data in which case all simulation should be included as part of `statistic`. Use of `sim = "parametric"` with a suitable `ran.gen` allows the user to implement any types of nonparametric resampling which are not supported directly.

Let's see an example:

Example #2 Parametric bootstrap using the `boot` function

The code is below. We first define the function to be passed to `rand.gen` that shows how to generate the data. `mle` is the list of needed parameter values (mean and standard deviation in this case). This is called `mle` in `boot` because these will often be maximum likelihood estimates. We use this function with `boot` as shown.

```
#Do parametric bootstrap for d1 data set:

#This is the function passed to rand.gen:
meangen<-function(dat,mle)
{  #function to generate random data set from normal(mle[[1]], mle[[2]]) distribution.
    return(rnorm(length(dat),mle[[1]],mle[[2]]))
}

bootmean<-function(dat) #function to generate test statistic
{  return(mean(dat))
}

#We don't really need to write a function to generate the test statistic
#in this case because the built in mean() function would work just fine.
#I put this here just to show that you often will need to write a wrapper
#function for your desired test statistic function

parboot<-boot(d2,bootmean,1000,ran.gen=meangen,mle=list(mean(d2),sd(d2)),sim="parametric")

boot.ci(parboot,type="perc") #Not that adjusted bootstrap method BCa does not work with parametric boot.

## BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
## Based on 1000 bootstrap replicates
##
## CALL :
## boot.ci(boot.out = parboot, type = "perc")
```

```
##
## Intervals :
## Level      Percentile
## 95%      ( 9.72, 10.57 )
## Calculations and Intervals on Original Scale
```

We again get very similar values for the CI.

Permutation Tests

Now we will switch gears and talk about hypothesis testing rather than confidence intervals. Suppose we have two samples

$$X_1 \dots X_n$$

$$Y_1 \dots Y_n$$

We wish to test the hypotheses

H_0 : the two samples come from the same distribution

H_1 : the distribution of the X sample is shifted relative to that of the Y sample.

We commonly conduct this hypothesis test as a two sample t-test. However, similarly to the case with a bootstrap, this is only valid if the data is approximately normally distributed (although the two sample t-test is quite robust to deviations from normality). What can we do if the data is not normally distributed?

Under H_0 , the data points are interchangeable between the two samples. That is, they come from the same distribution and switching data points between samples shouldn't change the sample properties. In this case, we can generate an approximate null distribution by the following procedure:

loop rep from 1 to R

{

1. Sample n values without replacement from $X_1 \dots X_n, Y_1 \dots Y_m$. These values will form new X sample. The remaining values will form the Y sample.

2. Calculate the test statistic for difference between the new X and Y samples. Store this value.

}

The collection of R test statistic values then gives an approximation to the null hypothesis distribution of the test statistic. We then compare the observed test statistic value to the value and reject H_0 if the observed value falls in the upper 5% of the permutation values. The permutation test p-value is

$$\hat{p} = \frac{\text{number of replicates with test-stat} > \text{observed value}}{\text{number of replicates}}$$

This is estimating the true p-value p . $\hat{p}$ is variable because the particular permutation samples that we get are variable. The p-value is the proportion of permutation replicates in which the value of the test statistic exceeds the value observed in the data. Similarly to the case with the bootstrap, we can view this proportion as a binomial proportion. The number of trials is equal to the number of permutation replicates and the probability of success is the true p-value. The 95% CI for the estimate of the p-value is just the 95% CI for a binomial proportion $\hat{p} \pm \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$, where n is the number of replicates and $\hat{p}$ is the estimate for the p-value.

example #1 - Permutations

Below is a program for a permutation test comparing two treatments:

```
#generate some data:
set.seed(836)
(d1<-rnorm(15,mean=3,sd=2))
```

```
## [1] 2.730197 2.855136 -1.063898 2.856210 3.211835 6.542038 4.801963
## [8] 4.569660 3.656039 4.029166 2.154168 5.112197 3.342328 3.296920
## [15] 3.294372
```

```
(d2<-rnorm(12,mean=2.5,sd=2))
```

```
## [1] 1.8454015 4.1273096 1.2173130 1.7374579 0.0204005 0.2928796 3.3413388
## [8] 1.9566117 3.4724822 3.6134028 3.6631222 1.6444504
```

```
ds<-c(d1,d2) #combined data set
```

```
#begin program_-----
```

```
l1<-length(d1) #sample sizes for data sets
l2<-length(d2)
```

```
#caculate observed difference in sample means (this will be our test statistic:
meandiffobs=(mean(d1)-mean(d2))
```

```
numperm<-10000 #number of permutations
meandiff<-numeric(numperm) #will contain permuted mean difference
```

```
for(i in 1:numperm) #loop over permutations
{
  s1<-sample(l1+l2,l1) #generate permutation sample
  perm1<-ds[s1] #values in s1
  perm2<-ds[-s1] #values not in s1
  meandiff[i]=(mean(perm1)-mean(perm2))
  #above is difference in the means for this permutation
}
#meandiff contains the test statistic values for each permutation
```

```
meandiff<-sort(meandiff,decreasing=T) #sort meandiff values
```

```
upperpval<-sum(meandiff>meandiffobs)/numperm #length(meandiff[meandiff>meandiffobs])/numperm #find perc
lowerpval<-sum(meandiff<meandiffobs)/numperm #find percentage less than observed
#p-value is one of above for one-sided
```

```
pval<-min(upperpval,lowerpval) #if two-sided, compare to 0.025
pval
```

```
## [1] 0.0275
```

Relationship to Wilcoxon Rank-Sum Test

The basic two sample permutation test is closely related to the Wilcoxon ranks-sum test. Under H_0 , any permutation is equally likely. Thus, the hypothesis test is posed as the question “was the mean difference in the observed permutation rare compared to the distribution of possible permutations?”. If we compared to an exhaustive list of permutations, then it would be exactly the same as the Wilcoxon rank-sum test. However, the number of possible permutations is very high (e.g. for this example $(25 \text{ choose } 12)=17,383,760$)

Exercise #3

- 1) Generate data sets of lengths 20 and 25 from a Poisson distribution with parameters 10 and 12, respectively.
- 2) Write code to conduct a permutation test of a difference between the distributions generating the data sets. Use the t-statistics as the test-stat.

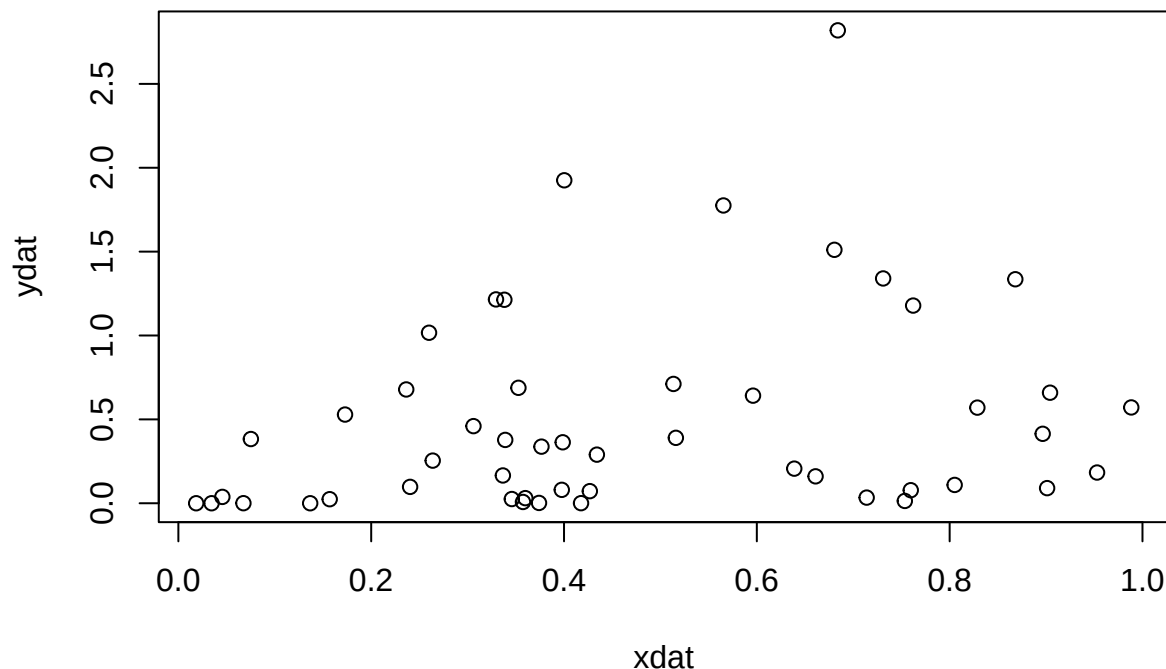
Permutation test for correlation

We can also do a permutation test for correlation. Under the null hypothesis there is no relationship between X and Y . Thus, any permutation of the Y -values relative to X -values is equally likely. We can generate a null hypothesis for the distribution of the correlation coefficient by randomly permuting Y -values relative to X -values.

First, generate some random data. This data does not come from a normal distribution and so the assumption of bivariate normality that is typically used in calculating p-values for correlation does not apply.

```
set.seed(3455454)
#generate data:
xdat<-runif(50)
ydat<-rgamma(50,shape=xdat,rate=1)

plot(xdat,ydat)
```



Next, test for significant correlation using permutations. The procedure is

```
loop over permutation replicates 1 to R
{
  randomly permute  $y$  values
  calculate the correlation coefficient between  $x$  and  $y$ .
}
```

The R permutation correlation coefficients approximate the null hypothesis distribution for the correlation coefficient. Calculate the p-value by comparing the observed correlation coefficient to the permutation values.

```
set.seed(12121)
corobs<-cor(xdat,ydat) #observed value of test statistic
numperm<-1000 #number of permutations
corperm<-numeric(numperm)

#bivariate normality does not apply, so can't use usual test
#for significant correlation.

pval<-sum(corperm>corobs)/numperm #proportion of permutation values greater than observed value
pval
```

```
## [1] 0
```

Here is the test based on bivariate normality:

```
cor.test(ydat,xdat)

##
## Pearson's product-moment correlation
##
## data: ydat and xdat
## t = 1.781, df = 48, p-value = 0.08125
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
## -0.03156886 0.49314003
## sample estimates:
## cor
## 0.2489666
```

This produces a p-value about twice the permutation value. Note that there is in reality a correlation between X and Y because we constructed the data that way.

Of course, we could also use spearman's rank correlation:

```
for(j in 1:numperm)
{corperm[j]<-cor(xdat,sample(ydat)) #sample(ydat) produces a random reshuffling of ydat
}

cor.test(xdat,ydat,method="spearman")
```

```
##  
## Spearman's rank correlation rho  
##  
## data:  xdat and ydat  
## S = 14012, p-value = 0.02082  
## alternative hypothesis: true rho is not equal to 0  
## sample estimates:  
##      rho  
## 0.3271549
```

This is preferable for simple correlation. However, permutation tests can be used in more complicated situations (e.g. we can test for interactions by permuting residuals)

A Note on Random Number Seeds

Throughout this lecture I have used `set.seed()` in my code for bootstrapping and permutations. This has the effect of making the random samples be the same every time - that is NOT random! I do this for demonstration purposes - so that my notes will be the same every time that I run the R Markdown. You will see that I do this frequently throughout my notes in this course. This is fine for presentation purposes, but you should NOT do this in any real data analysis or simulations. If you set the seed, then you will get the same results every time. Those results may be a fluke (i.e. due to random good or bad luck). You must let the random number generators be random in a real analysis.